
Snake évolutif sur grille dynamique

Conception d'un agent neuroévolutionnaire
dans un environnement Gymnasium personnalisé

Projet final — Sujet 2

Élément de module : M2442 « Jeux Vidéo IA »

Membres de l'équipe

Benaqqa Moubarak

Bensmina Anass

Table des matières

1	Introduction	2
1.1	Contexte et motivation	2
1.2	Objectifs	2
1.3	Organisation du rapport	2
2	Le jeu comme système interactif et environnement Gymnasium	2
2.1	Snake vu comme système interactif	2
2.2	Description du jeu	3
2.3	Architecture « Entrées – Traitement – Sorties »	3
2.4	Boucle de jeu et discrétisation du temps	3
2.5	État global et observation locale ($o = \varphi(s)$)	4
2.6	Espace d'actions	4
2.7	Règles, transition et dynamique	4
2.8	Objectifs et fonction de récompense	5
2.9	Conditions de fin et conformité à l'API Gymnasium	5
2.10	Grille dynamique	5
3	Agent et algorithme génétique	5
3.1	Politique : réseau de neurones	6
3.2	Génome	6
3.3	Fonction de fitness	6
3.4	Opérateurs génétiques	6
3.5	Boucle d'évolution	7
4	Protocole expérimental	7
4.1	Hyperparamètres	7
4.2	Baselines de comparaison	7
4.3	Jeux de données et reproductibilité	7
4.4	Indicateurs quantitatifs	7
5	Résultats et discussion	8
5.1	Dynamique de l'évolution	8
5.2	Comparaison aux baselines	8
5.3	Distribution des scores	9
5.4	Étude de robustesse	9
5.5	Limites	10
5.6	Pistes d'amélioration	10
6	Conclusion	10

1. Introduction

Ce rapport présente la conception, l'implémentation et l'évaluation expérimentale d'un agent évolutionnaire jouant à une variante du jeu *Snake* dans un environnement à difficulté variable. Le travail s'inscrit dans le cadre du projet final du module *M2442 – Jeux Vidéo IA* et correspond au **Sujet 2 : Snake évolutif sur grille dynamique**, dont l'énoncé demande de « créer un mini-jeu de type Snake avec nourriture, obstacles et conditions variables selon les épisodes », au moyen d'un « environnement Gymnasium personnalisé » et d'un « agent évolutionnaire maximisant survie et score ».

1.1 Contexte et motivation

Les algorithmes génétiques (AG) constituent une famille de méthodes d'optimisation inspirées de l'évolution naturelle. Contrairement à l'apprentissage par renforcement classique (qui repose sur le gradient et la rétropropagation temporelle de la récompense), un AG explore l'espace des politiques par sélection, croisement et mutation, sans calcul de gradient. Cette approche, dite *neuroévolution* lorsqu'on optimise les poids d'un réseau de neurones, est particulièrement adaptée aux environnements de jeu où la fonction de récompense est éparse, bruitée ou non différentiable.

Le jeu Snake offre un terrain d'expérimentation idéal : règles simples, espace d'états combinatoire, et tension naturelle entre deux objectifs (manger pour augmenter le score, mais survivre en évitant murs, obstacles et propre corps).

1.2 Objectifs

- Concevoir un environnement Gymnasium personnalisé respectant explicitement l'API (observations, actions, récompense, fin d'épisode).
- Introduire une **dynamique inter-épisodes** : densité d'obstacles et budget de survie variables d'un épisode à l'autre.
- Implémenter un agent neuroévolutionnaire (réseau de neurones optimisé par AG) et définir une fonction de fitness pertinente.
- Comparer cet agent à au moins une **baseline** sur un protocole expérimental reproductible.
- Analyser de manière critique les performances, les limites et les pistes d'amélioration.

1.3 Organisation du rapport

La section 2, la plus développée, modélise le jeu *comme système interactif* et détaille la conception de l'environnement Gymnasium. La section 3 décrit l'agent et l'algorithme génétique. La section 4 expose le protocole expérimental. La section 5 présente et discute les résultats. La section 6 conclut.

2. Le jeu comme système interactif et environnement Gymnasium

Avant d'aborder l'agent, nous modélisons rigoureusement le jeu lui-même. Un jeu vidéo est un **système numérique interactif** dans lequel un ou plusieurs agents effectuent des actions qui modifient un monde simulé régi par des règles, le système renvoyant un *feedback* (visuel, score) orienté vers un objectif. Concevoir le Sujet 2 revient donc d'abord à construire ce système — sa boucle, son état, ses règles, son rendu — puis à y brancher un agent. Cette section suit cette logique d'ingénierie du jeu.

2.1 Snake vu comme système interactif

On peut décomposer notre jeu selon les cinq éléments incontournables d'un système interactif :

Élément	Réalisation dans le projet
Agent	Le serpent, piloté par une politique (réseau de neurones évolué).
Environnement	Grille 12×12 , obstacles fixes, nourriture, corps du serpent.
Actions	Tout droit / tourner à droite / tourner à gauche (espace discret).
Règles	Déplacement case par case, collisions, ingestion, faim, victoire.
Feedback	Observation, récompense numérique, rendu (texte ANSI ou Pygame).

En résumé : *état* = position du serpent, des obstacles et de la nourriture ; *action* = changement de direction ; *règle* = collision mur/corps ; *objectif* = score (nourriture mangée).

2.2 Description du jeu

Le serpent évolue sur une grille carrée de $N \times N$ cases ($N = 12$ dans nos expériences). Il se déplace case par case dans la direction courante. À chaque pas de temps, l'agent choisit une action ; le serpent avance, et :

- s'il atteint la **nourriture**, il grandit d'une case, le score augmente de 1, et une nouvelle nourriture apparaît sur une case libre ;
- s'il heurte un **mur**, un **obstacle** ou son **propre corps**, l'épisode se termine (collision) ;
- s'il ne mange pas pendant un nombre de pas supérieur au **budget de faim**, l'épisode se termine (famine).

2.3 Architecture « Entrées – Traitement – Sorties »

L'environnement suit le modèle fonctionnel classique d'un moteur de jeu : il *reçoit* une entrée (l'action de l'agent), *traite* cette entrée (mise à jour de l'état selon les règles et la physique du déplacement), puis *produit* des sorties (nouvel état perçu, récompense, rendu). Le tableau ci-dessous situe nos modules de code par rapport aux composants d'un moteur de jeu généraliste.

Composant moteur	Rôle	Implémentation (snake_evo)
Gestion du monde	Carte, entités, état	SnakeEnv.reset (serpent, obstacles, nourriture)
Physique / mouvement	Déplacement, avance	SnakeEnv.step (insertion tête / retrait queue)
Collisions	Détection des contacts	_is_collision (murs, obstacles, corps)
Règles / scoring	Manger, faim, victoire	logique de step + calcul de récompense
Perception (IA)	Observation → décision	_get_obs + MLPPolicy
Rendu	Sortie perceptive	render (ANSI) / render_pygame

On notera que le **rendu est volontairement découplé** de la simulation : le renderer Pygame se contente de lire l'état public de l'environnement. Le « rendu » n'est pas la simulation, c'est sa représentation ; on peut donc entraîner sans aucun affichage (mode rapide) puis rejouer une partie graphiquement à l'identique.

2.4 Boucle de jeu et discrétisation du temps

Le jeu est un système dynamique exécuté par **itérations**. Le temps est *échantillonné en pas discrets* $t_0, t_1, \dots$ et l'état évolue selon une dynamique de transition

$$s_{t+1} = f(s_t, a_t),$$

où s_t est l'état courant et a_t l'action. Le jeu étant *au tour par tour* (et non en temps réel), il n'y a pas de pas de temps physique Δt variable ni de budget de *frame* : chaque appel à **step** correspond exactement à une transition. La boucle d'interaction agent–environnement s'écrit :

```

1  obs, info = env.reset(seed=...)          # Input initial : etat de depart
2  done = False
3  while not done:                          # Boucle de jeu (par tour)
4      action = agent.act(obs)              # Decision : observation -> action
5      obs, reward, terminated, truncated, info = env.step(action) # Update +
                                         regles
6      done = terminated or truncated        # Condition d'arret

```

Ce schéma correspond au cycle *Perception* → *Décision* → *Action* (PDA) : l'agent *observe* (obs), *décide* (act), puis le moteur *met à jour* l'état et renvoie le *feedback*.

2.5 État global et observation locale ($o = \varphi(s)$)

Il faut distinguer l'**état** et l'**observation**, conformément au cadre du cours. L'*état global* s_t contient toute l'information nécessaire à la simulation : positions de *tout* le corps du serpent, direction courante, ensemble des obstacles, position de la nourriture, compteur de faim et score. L'*observation* $o_t = \varphi(s_t)$ est seulement ce que l'agent perçoit : un vecteur compact de **11 composantes** à valeurs dans $[-1, 1]$, adapté à un contrôleur de type réseau de neurones.

Indices	Signification
0–2	Danger immédiat (tout droit / droite / gauche)
3–6	Direction courante (encodage <i>one-hot</i> : haut/droite/bas/gauche)
7–10	Position relative de la nourriture (gauche/droite/haut/bas)

La fonction de perception φ est donc une *compression* de l'état global en une représentation **égocentrique et locale** (« qu'est-ce qui est dangereux autour de moi, où est la nourriture ? »). Ce choix de conception a trois vertus : réduire la dimension d'entrée de la décision, favoriser la *généralisation* (l'agent ne mémorise pas une carte précise), et rendre le problème *partiellement observable* de façon réaliste — l'agent ne « voit » pas l'intégralité de la grille, exactement comme une IA de jeu dotée d'un champ de perception limité. Le revers (analysé en section 5) est que cette myopie empêche d'anticiper les pièges à plusieurs cases.

2.6 Espace d'actions

L'espace d'actions est **discret** et contient 3 actions *relatives* au serpent : aller *tout droit*, *tourner à droite*, *tourner à gauche*. Un bon espace d'action doit être *expressif* (permettre les comportements voulus), *contrôlable* (éviter l'explosion combinatoire), *aligné* avec la dynamique, et *stable*. Notre choix relatif à 3 actions satisfait ces critères : plus simple qu'un espace absolu à 4 directions, il **élimine mécaniquement les demi-tours suicidaires** (impossibles à exprimer) et réduit la dimension du problème, conformément au cadrage « espace d'actions discret simple » imposé par l'énoncé.

2.7 Règles, transition et dynamique

Les **règles** définissent ce que « fait » la fonction de transition f : les effets des actions, les interactions (collisions), et les conditions d'arrêt. Concrètement, à chaque **step**, le moteur : (i) tourne la direction selon l'action, (ii) calcule la nouvelle tête, (iii) teste la collision (mur, obstacle, corps), (iv) gère l'ingestion (croissance + nouvelle nourriture) ou l'avance simple (retrait de la queue), (v) vérifie la faim et la victoire.

Concernant la nature de la dynamique : la transition est **déterministe** à l'intérieur d'un *épisode* — pour un même état et une même action, le résultat est toujours identique, ce qui garantit la reproductibilité de l'évaluation, $P(s_{t+1} | s_t, a_t) \in \{0, 1\}$. La **stochasticité** est injectée uniquement au **reset** (placement aléatoire des obstacles et de la nourriture, tirage du nombre d'obstacles), via un générateur aléatoire ensemencé par une graine. Cette séparation est un choix

d'ingénierie important : elle permet un aléa *contrôlé* et *rejouable* (deux épisodes de même graine sont identiques au pixel près), indispensable pour comparer équitablement les agents.

2.8 Objectifs et fonction de récompense

L'**objectif** du jeu est de maximiser le score (nourriture mangée) tout en survivant. La **récompense** $r_t = R(s_t, a_t, s_{t+1})$ est un signal numérique qui mesure la qualité d'un comportement ; elle combine ici l'objectif principal et un *shaping* léger :

$$r = \begin{cases} +1.0 & \text{si la nourriture est mangée,} \\ -1.0 & \text{en cas de collision (mort),} \\ -0.5 & \text{en cas de famine,} \\ +0.1 & \text{si l'agent se rapproche de la nourriture,} \\ -0.15 & \text{s'il s'en éloigne,} \\ -0.01 & \text{coût de temps par pas,} \end{cases}$$

la victoire (grille entièrement remplie) octroyant un bonus de +5. Le *shaping* de distance accélère l'apprentissage en fournissant un signal dense plutôt qu'une récompense éparse.

Le piège de la récompense « trichable ». Un principe de conception central est qu'il faut *récompenser ce que l'on veut réellement obtenir*, et non un indicateur ambigu. Deux écueils nous menaçaient : récompenser la seule survie inciterait au *camping* (l'agent ne fait rien d'utile), et récompenser la seule proximité à la nourriture inciterait à *tourner en rond* près d'elle. Nous mitigeons ces dérives par le faible **coût de temps** par pas (qui pénalise les boucles stériles) et surtout par la **domination du score** ($\times 100$) dans la fitness évolutionnaire (section 3.3), de sorte que la survie ne soit qu'un bonus secondaire.

2.9 Conditions de fin et conformité à l'API Gymnasium

L'environnement implémente strictement l'API `gymnasium.Env`. L'espace d'actions est déclaré `spaces.Discrete(3)` et l'espace d'observations `spaces.Box(-1, 1, shape=(11,))`. La méthode `step` renvoie le quintuplet standard (`obs`, `reward`, `terminated`, `truncated`, `info`) : un épisode se *termine* (`terminated`) en cas de collision, de famine ou de victoire ; il est *tronqué* (`truncated`) au-delà d'un nombre maximal de pas fixé par le protocole. Le dictionnaire `info` expose des variables d'état utiles (score, nombre d'obstacles, cause de fin) pour l'analyse, sans servir à la décision. Cette conformité garantit que l'environnement est interchangeable avec n'importe quel outil de l'écosystème Gymnasium.

2.10 Grille dynamique

La spécificité du Sujet 2 est que les conditions varient *selon les épisodes*. À chaque `reset`, l'environnement tire aléatoirement :

- le nombre d'obstacles fixes, dans l'intervalle $[3, 10]$;
- le budget de faim, à $\pm 20\%$ de sa valeur de base N^2 .

Les obstacles sont placés aléatoirement sur la grille (en évitant le corps initial du serpent et la zone immédiatement devant sa tête). Cette variabilité empêche l'agent de mémoriser une carte fixe et l'oblige à apprendre une *politique générale* robuste à la configuration — c'est l'incertitude événementielle qui distingue un vrai jeu d'un simple programme déterministe.

3. Agent et algorithme génétique

Dans la taxonomie des agents de jeu (scripté $\rightarrow$ réactif $\rightarrow$ délibératif $\rightarrow$ adaptatif $\rightarrow$ **évolutif/apprenant**), notre agent se situe à l'extrémité *évolutive* : sa politique n'est pas codée à la

main mais *optimisée hors ligne* par recherche évolutionnaire. Il s’oppose ainsi directement à notre baseline *réactive* (l’heuristique, à base de règles **si/alors**). Cette section décrit la politique (le contrôleur π), son génome, et l’algorithme qui le fait évoluer.

3.1 Politique : réseau de neurones

La politique de l’agent est un **perceptron multicouche** (MLP) :

$$\text{obs}_{(11)} \xrightarrow{W_1, b_1} \tanh \xrightarrow{(16)} \xrightarrow{W_2, b_2} \text{logits}_{(3)} \xrightarrow{\arg \max} \text{action}.$$

L’entrée est le vecteur d’observation (11), suivie d’une couche cachée de 16 neurones à activation tanh, puis d’une couche de sortie produisant un score par action. L’action retenue est l’**argmax** (politique *déterministe*, ce qui garantit la reproductibilité de l’évaluation).

3.2 Génome

Le **génome** d’un individu est le vecteur aplati de tous les paramètres du réseau :

$$\dim(\text{génome}) = \underbrace{11 \times 16}_{W_1} + \underbrace{16}_{b_1} + \underbrace{16 \times 3}_{W_2} + \underbrace{3}_{b_2} = \mathbf{243}.$$

Évoluer ces 243 réels revient à chercher la meilleure politique dans un espace continu de dimension 243.

3.3 Fonction de fitness

La fitness d’un génome est évaluée sur plusieurs épisodes (graines fixées au sein d’une même génération, pour une comparaison équitable). En notant $\bar{s}$ le score moyen (nourriture mangée), $\bar{t}$ le nombre de pas moyen et $\bar{R}$ la récompense cumulée moyenne :

$$\text{fitness} = 100 \bar{s} + 0.5 \bar{t} + \bar{R}.$$

La nourriture domine le critère ($\times 100$), conformément à l’objectif « maximiser score et survie » : la survie ($\bar{t}$) n’intervient que comme bonus secondaire, afin d’éviter de récompenser un serpent qui survit sans jamais manger.

3.4 Opérateurs génétiques

Sélection

par **tournoi** de taille 5 : on tire 5 individus au hasard et on retient le meilleur. Cela maintient une pression de sélection modérée tout en préservant la diversité.

Croisement

uniforme : chaque gène de l’enfant provient aléatoirement de l’un des deux parents (probabilité 0.7 d’appliquer le croisement, sinon clonage du premier parent).

Mutation

gaussienne additive : chaque gène est perturbé avec probabilité 0.1 par un bruit $\mathcal{N}(0, \sigma^2)$. L’amplitude σ décroît linéairement de 0.25 à 0.10 au fil des générations (exploration forte au début, affinage en fin).

Élitisme

les 12% meilleurs individus sont copiés tels quels dans la génération suivante, garantissant que la meilleure solution n’est jamais perdue.

3.5 Boucle d'évolution

```

1 for gen in range(n_generations):
2     # 1. Évaluer la fitness de chaque individu (multi-épisodes)
3     fitnesses = [evaluate_genome(ind, ...) for ind in population]
4     # 2. Conserver les meilleurs (élitisme)
5     new_pop = elites(population, fitnesses)
6     # 3. Reproduire jusqu'à remplir la population
7     while len(new_pop) < pop_size:
8         p1, p2 = tournoi(fitnesses), tournoi(fitnesses)
9         enfant = mutation(croisement(p1, p2))
10        new_pop.append(enfant)
11    population = new_pop

```

4. Protocole expérimental

4.1 Hyperparamètres

Paramètre	Valeur	Paramètre	Valeur
Taille de grille	12 × 12	Taille de population	150
Obstacles (dynamique)	[3, 10]	Nombre de générations	150
Couche cachée	16 neurones	Fraction d'élite	12 %
Taille du génome	243	Taille de tournoi	5
Taux de mutation	0.10	Croisement	0.70
σ mutation	0.25 → 0.10	Épisodes d'éval./ind.	6
Pas max / épisode	400	Graine AG	42

4.2 Baselines de comparaison

Conformément à l'exigence d'au moins une baseline, nous en utilisons deux :

- **Agent aléatoire** : choisit une action uniformément au hasard. Borne inférieure de performance.
- **Agent heuristique** : règle programmée à la main — parmi les actions sans danger immédiat (lu dans l'observation), choisir celle qui rapproche de la nourriture. Baseline « forte » servant de référence exigeante.

4.3 Jeux de données et reproductibilité

Point méthodologique central : les graines utilisées pour **évaluer** l'agent final sont *disjointes* de celles vues pendant l'**entraînement**. Les épisodes de test correspondent aux graines [900000, 900199], jamais rencontrées durant l'évolution. Tous les agents (AG et baselines) sont évalués sur **exactement les mêmes 200 épisodes de test**, ce qui garantit une comparaison équitable et reproductible. L'ensemble des graines, hyperparamètres et résultats est sauvegardé au format JSON.

4.4 Indicateurs quantitatifs

Pour chaque agent nous reportons : le score moyen et son écart-type, le score maximal, la survie moyenne (nombre de pas), et le taux de réussite (proportion d'épisodes avec au moins une nourriture mangée). Une étude de **robustesse** mesure en outre le score selon une densité d'obstacles fixée (0, 4, 8, 12).

5. Résultats et discussion

5.1 Dynamique de l'évolution

La figure 1 montre l'évolution de la fitness et du score au fil des 150 générations. On observe une progression nette : le score du meilleur individu passe d'environ 0.3 à la première génération à des pics dépassant 13 en fin d'évolution, tandis que le score moyen de la population croît de façon régulière de ≈ 0 à ≈ 6 . La meilleure fitness atteinte est de 1707.7. Le bruit résiduel des courbes provient du renouvellement des graines d'évaluation à chaque génération (chaque génération affronte des cartes différentes), ce qui constitue une mesure de généralisation plutôt que de surapprentissage.

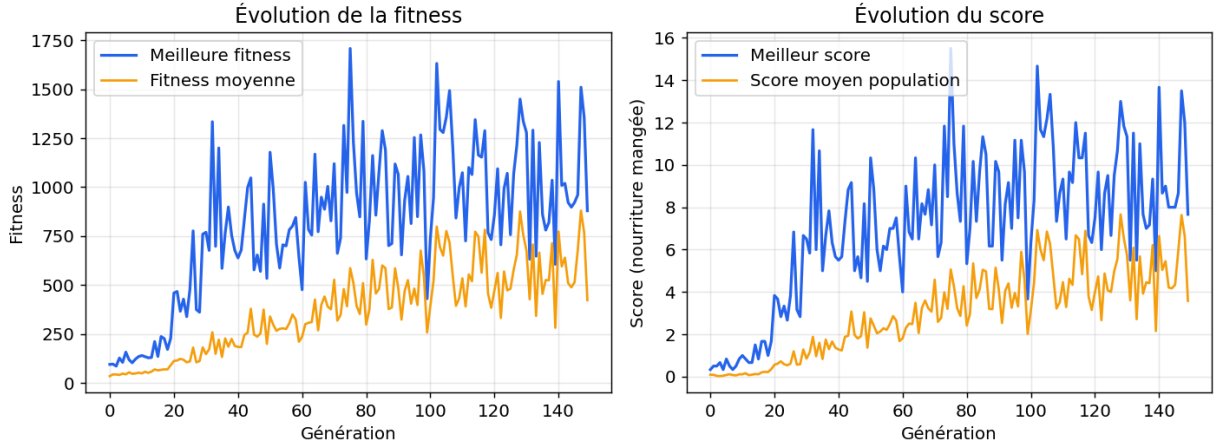


FIGURE 1 – Évolution de la fitness (gauche) et du score (droite) au fil des générations. Bleu : meilleur individu ; orange : moyenne de la population.

5.2 Comparaison aux baselines

Le tableau 1 et la figure 2 résument les performances sur les 200 épisodes de test indépendants.

TABLE 1 – Performances sur 200 épisodes de test (grille dynamique, graines [900000, 900199]).

Agent	Score moy.	Écart-type	Score max	Survie moy.	Taux succès
Aléatoire	0.09	0.30	2	15.4	8.5 %
AG (neuroévolution)	6.30	6.20	25	172.8	91.5 %
Heuristique (réf.)	15.97	6.96	36	153.1	100 %

Interprétation. L'agent neuroévolutionnaire améliore le score moyen d'un facteur ~ 70 par rapport à l'agent aléatoire (6.30 contre 0.09), ce qui démontre sans ambiguïté que l'évolution a découvert une politique compétente à *partir de poids aléatoires*, sans aucune connaissance programmée du jeu. Fait notable, l'agent AG **survit en moyenne plus longtemps** que l'heuristique (172.8 pas contre 153.1) : il a appris une gestion prudente de l'espace. Il atteint un score maximal de 25 sur une partie.

En revanche, l'AG reste *en deçà* de l'heuristique en score moyen (6.30 contre 15.97). Ce résultat est attendu et instructif : l'heuristique bénéficie d'une logique d'évitement de danger *codée à la main* et toujours optimale localement, que l'AG doit au contraire *découvrir* dans un espace de 243 paramètres avec un budget limité de 150 générations.

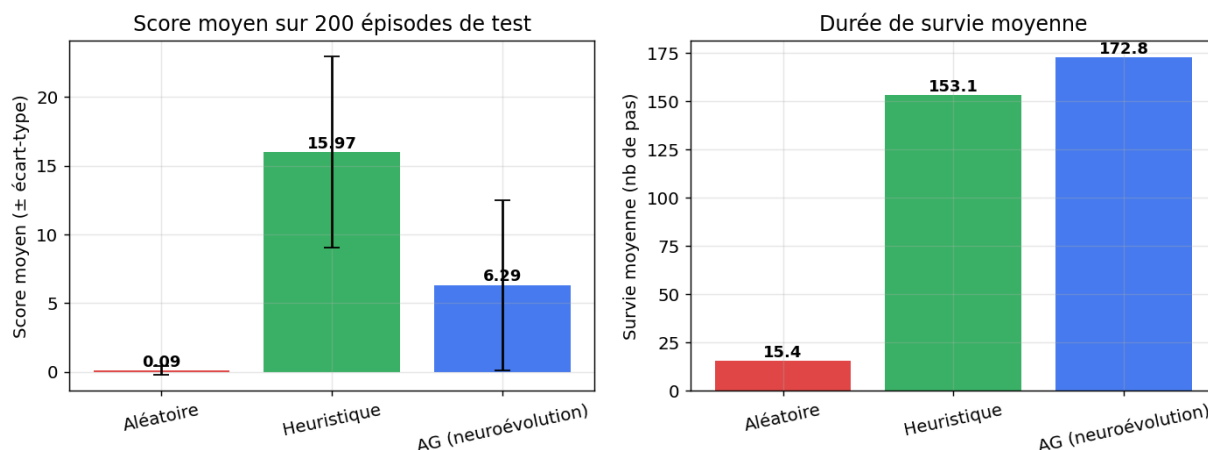


FIGURE 2 – Score moyen (gauche, $\pm$ écart-type) et durée de survie moyenne (droite) des trois agents sur le jeu de test.

5.3 Distribution des scores

La figure 3 compare les distributions de scores. L'AG présente une distribution étalée : beaucoup d'épisodes à score faible (parties difficiles, forte densité d'obstacles) mais une queue droite atteignant des scores élevés. La médiane de l'AG est de 4 contre 16 pour l'heuristique.

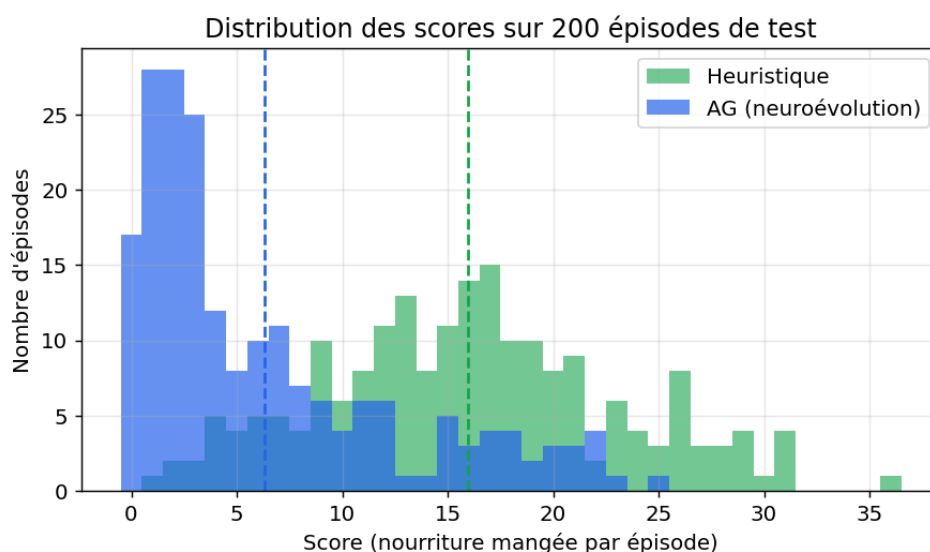


FIGURE 3 – Distribution des scores sur les 200 épisodes de test. Les lignes pointillées indiquent les moyennes.

5.4 Étude de robustesse

La figure 4 révèle un résultat éclairant. **Sur grille sans obstacle**, l'AG (18.93) rivalise presque avec l'heuristique (19.85) : la politique apprise de recherche de nourriture est donc excellente. La performance de l'AG chute cependant rapidement avec la densité d'obstacles (9.6 à 4 obstacles, 2.5 à 12), beaucoup plus vite que l'heuristique. L'écart global constaté précédemment s'explique donc essentiellement par une **gestion sous-optimale des obstacles**, et non par une mauvaise recherche de nourriture.

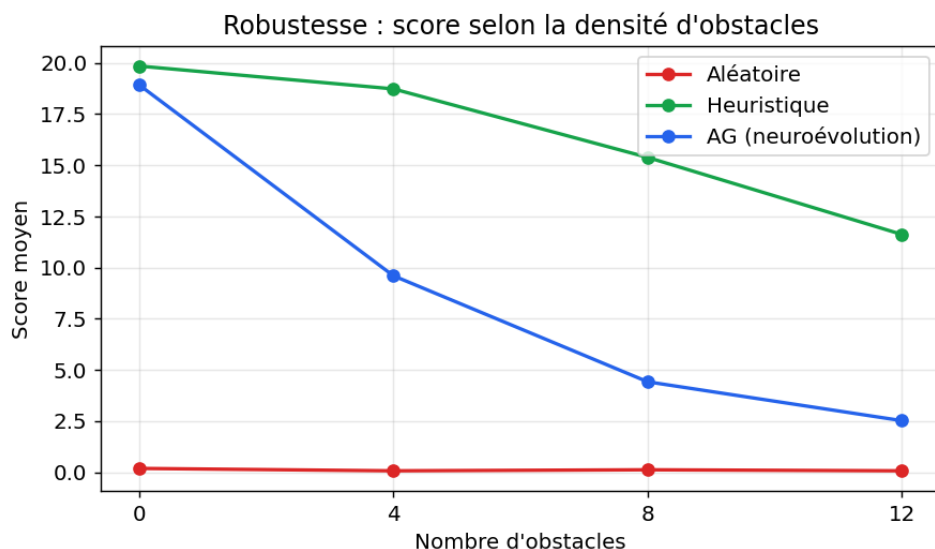


FIGURE 4 – Score moyen selon la densité d'obstacles (grille statique). L'AG égale presque l'heuristique sans obstacle mais se dégrade plus vite.

5.5 Limites

- **Observation locale** : la perception ne « voit » que le danger immédiat (une case). Le serpent ne peut pas anticiper les pièges à plusieurs cases ni les impasses, ce qui explique sa fragilité face aux obstacles denses et à son propre corps lorsqu'il s'allonge.
- **Coût d'évaluation** : la fitness étant bruitée (6 épisodes aléatoires), un bon individu peut être sous-évalué par malchance, ralentissant la convergence.
- **Capacité du réseau** : 16 neurones cachés et 243 paramètres limitent la complexité de la politique apprenable.
- **Budget évolutif** : 150 générations restent modestes pour la neuroévolution ; les courbes ne sont pas encore saturées.

5.6 Pistes d'amélioration

- **Enrichir l'observation** : ajouter une vision sur plusieurs cases (rayons de distance aux dangers), la longueur du serpent, ou une carte locale, pour permettre l'anticipation.
- **Réduire le bruit de fitness** : augmenter le nombre d'épisodes d'évaluation ou utiliser des graines communes (CRN, *common random numbers*).
- **Opérateurs avancés** : mutation auto-adaptative, ou stratégies d'évolution modernes (CMA-ES) qui adaptent la covariance.
- **Topologie évolutive** (NEAT) : faire évoluer aussi la structure du réseau et non seulement ses poids.
- **Curriculum** : commencer avec peu d'obstacles puis augmenter progressivement la difficulté.

6. Conclusion

Nous avons conçu, de bout en bout, un environnement Gymnasium personnalisé pour une variante *dynamique* du jeu Snake (Sujet 2), puis un agent neuroévolutionnaire dont les 243 poids d'un MLP sont optimisés par un algorithme génétique complet (sélection par tournoi, croisement uniforme, mutation gaussienne à amplitude décroissante, élitisme). Le protocole expérimental, reproductible et fondé sur une séparation stricte entre épisodes d'entraînement et de test, montre que l'agent évolué apprend une politique nettement supérieure au hasard (score moyen $\times 70$) et survit même plus longtemps que la baseline heuristique.

L'analyse critique met en évidence que l'écart résiduel avec l'heuristique provient surtout d'une gestion perfectible des obstacles, conséquence directe d'une observation purement locale — un diagnostic confirmé par l'étude de robustesse, où l'agent égale presque l'heuristique en l'absence d'obstacle. Ces limites ouvrent des pistes d'amélioration concrètes (perception enrichie, réduction du bruit de fitness, stratégies d'évolution avancées, curriculum), qui constitueraient la suite naturelle de ce travail.

Au-delà des chiffres, ce projet illustre l'intérêt de la neuroévolution comme alternative sans gradient pour l'apprentissage de comportements de jeu : à partir d'une initialisation purement aléatoire et sans aucune connaissance experte injectée, l'évolution fait émerger une stratégie cohérente de recherche de nourriture et de survie.

Reproductibilité. Tout le code (environnement, agents, algorithme génétique, scripts d'entraînement et de figures) est fourni dans l'archive du projet. L'expérience complète se relance par `python experiments/train.py` puis `python experiments/make_figures.py`.